

Object-Oriented Programming in Java

Emily Lucia Antosch

17.09.2026

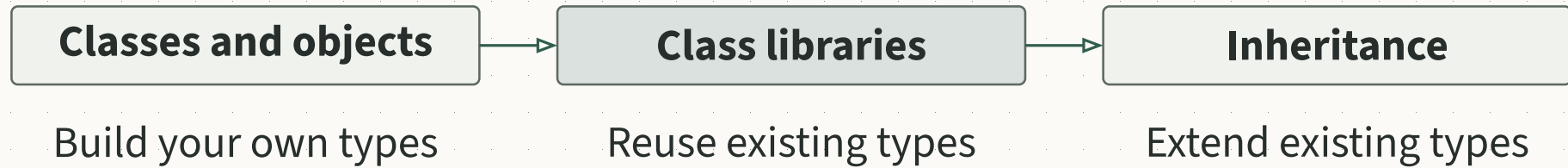
Today's route

1. Introduction	2
2. Strings	5
3. Arrays	30
4. Multidimensional Arrays	46
5. Lists	57
6. foreach Loop	62
7. Wrapper Classes & Math Class	69
8. License Notice	77

1. Introduction

Where Are We Now?

Today



The Goal of This Chapter

- Create, compare, and format strings.
- Organize data in arrays, matrices, and lists.
- Convert text to numbers and use mathematical functions.

2. Strings

Strings: C and Java

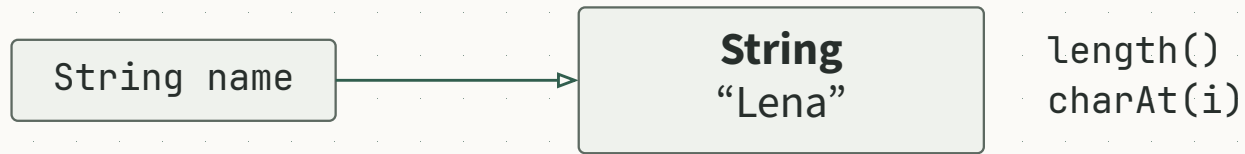
C

Characters end at `\0`



Program manages storage and bounds

Java



Reference to object · runtime manages memory

Creating Strings

Prefer a literal. Use new when you need a distinct String object.

```
1 String name = "Lena";  
2 String copy = new String("Lena");
```

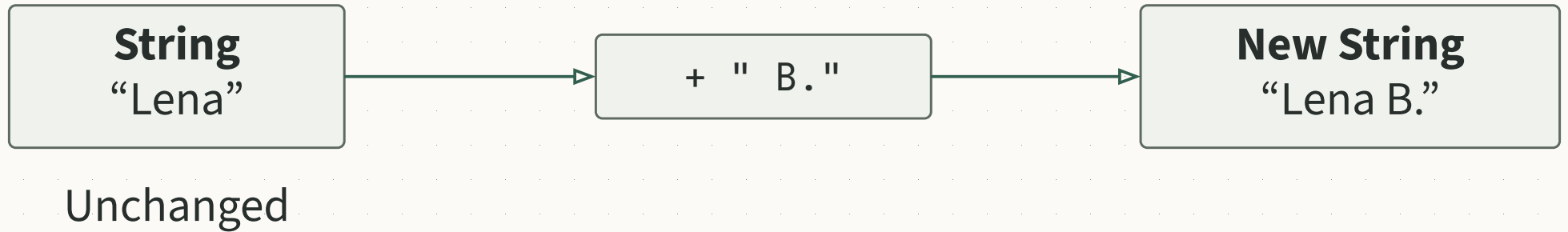
Java

Reassignment changes the reference, not the original String.

```
1 name = "Birgit";
```

Java

Strings Are Immutable



StringBuilder: build text in several steps

Strings Are Immutable



What will this print?

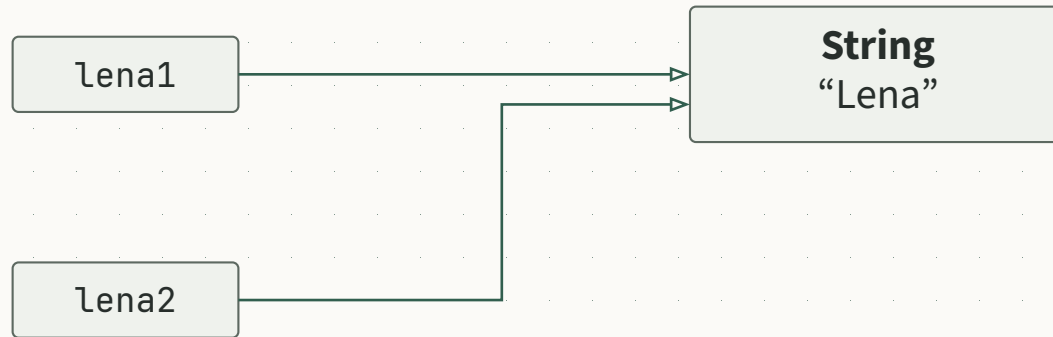
```
1 String lena1 = new String("Lena");  
2 String lena2 = lena1;  
3 System.out.println(lena1 + " / " + lena2);  
4 System.out.println(lena1 == lena2);  
5  
6 lena2 += " B.";  
7 System.out.println(lena1 + " / " + lena2);  
8 System.out.println(lena1 == lena2);
```

Java

Strings Are Immutable

```
1 String lena1 = new String("Lena");  
2 String lena2 = lena1;
```

Java



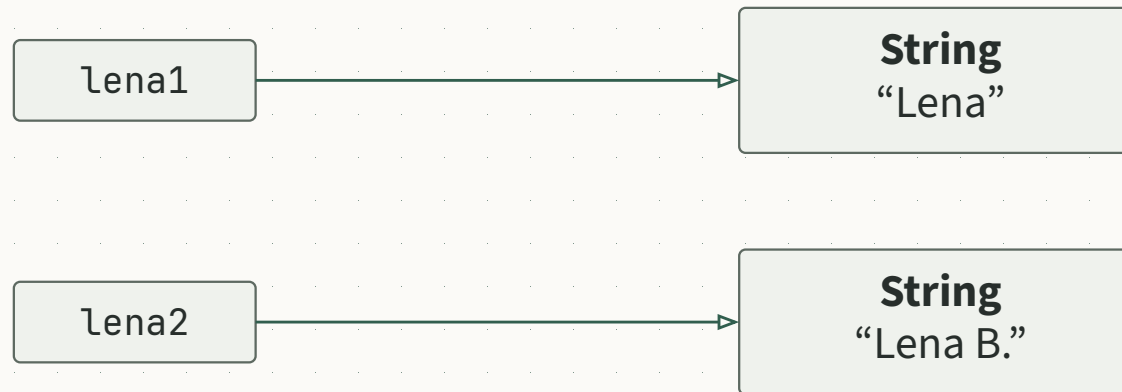
Two references, one immutable object

`lena1 == lena2 → true`

Strings Are Immutable

```
1 lena2 += " B.";
```

Java



New object; the original String is unchanged

`lena1 == lena2` → false

String Concatenation

Use + to concatenate text; += reassigns the result.

```
1 String name = "Lena " + "or " + "then ";  
2 name = name + "after all " + "again ";  
3 name += "Birgit?";
```

Java

String Concatenation



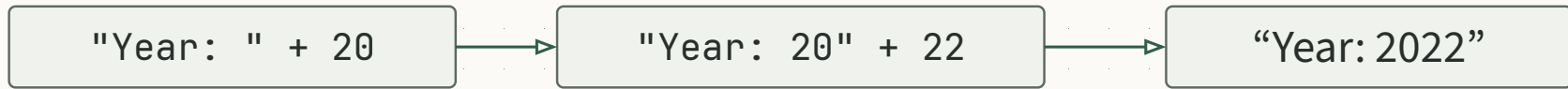
What will this print?

```
1  int a = 20;  
2  int b = 22;  
3  System.out.println("Year: " + a + b);  
4  System.out.println(a + b + " (Year)");
```

Java

String Concatenation

Evaluate + from left to right



If either operand is a String, + concatenates

The toString() Method



`public String toString()`

Concatenation result: `"Name: Lena Jensen"`

The toString() Method



Task 1

Create a Person class with `toString()`. Test its implicit call in console output.

The toString() Method

```
1  public class Person {  
2      String firstName, surname;  
3  
4      public Person(String firstName, String surname) {  
5          this.firstName = firstName;  
6          this.surname = surname;  
7      }  
8  
9      public String toString() {  
10         return firstName + " " + surname;  
11     }  
12 }
```

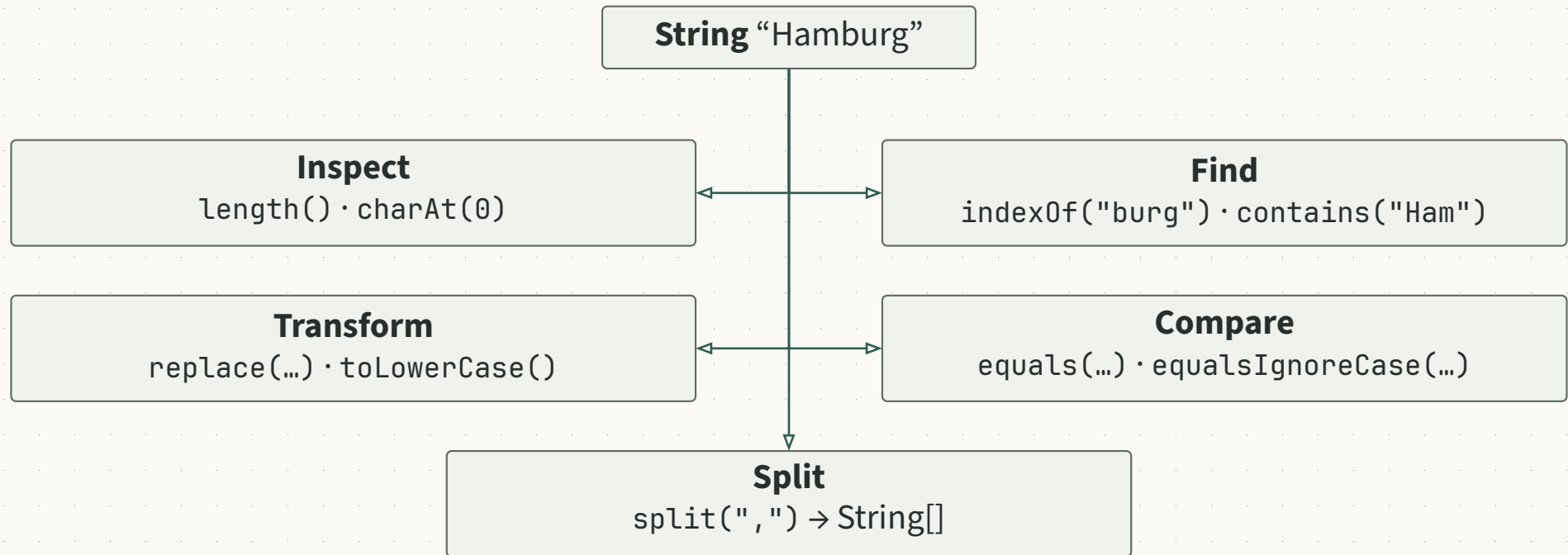
Java

The toString() Method

```
1 Person lena = new Person("Lena", "Jensen");  
2 String name = "Name: " + lena;  
3 System.out.println(lena);  
4 System.out.println(name);
```

Java

String Methods



`charAt(0)` returns the first character. Indices start at zero.

String Methods



Task 2

Replace “Humbug” with “Hamburg”.

```
1 String hamburg = "Welcome to Humbug!";  
2 hamburg = hamburg.replace("Humbug", "Hamburg");  
3 System.out.println(hamburg);
```

Java

String Methods



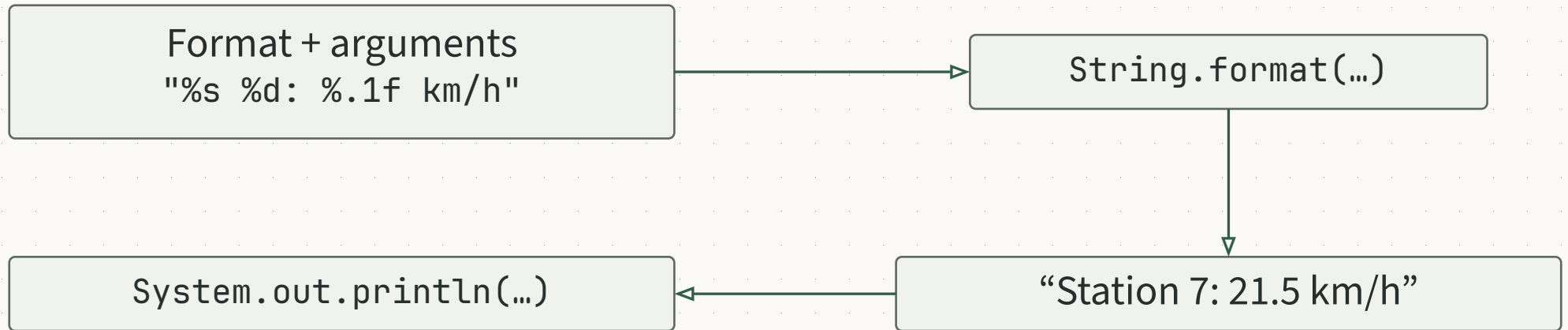
What will this print?

```
1 String upper = "Welcome to Hamburg!";  
2 String lower = "welcome to hamburg!";  
3  
4 System.out.println(lower.equals(upper));  
5 System.out.println(lower.equals(upper.toLowerCase()));  
6 System.out.println(lower.equalsIgnoreCase(upper));
```

Java

`equals` compares contents; `=` compares references.

String Formatting



Formatting returns text; println prints it

Java format strings use syntax similar to C's printf.

String Formatting



What will this print?

```
1 import java.util.Locale;  
2 // Inside main():  
3 double wind = 21.4532;  
4 String weather = String.format(Locale.US,  
5     "%s %d: %.1f km/h", "Station", 7, wind);  
6 System.out.println(weather);
```

Java

String Formatting



What will this print?

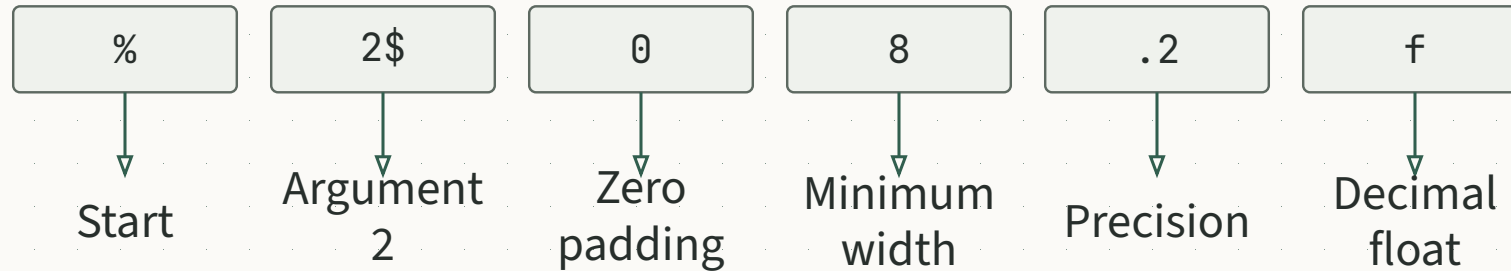
```
1 import java.util.Locale;  
2 // Inside main():  
3 double wind = 21.4532;  
4 String weather = String.format(Locale.US,  
5     "%s %d: %.1f km/h", "Station", 7, wind);  
6 System.out.println(weather);
```

Java

Output: Station 7: 21.5 km/h

String Formatting

`%[argument_index$][flags][width][.precision]conversion`



Only % and the conversion are required

String Formatting

Conversion	Meaning
f, e, g	Floating point
d	Decimal integer
o	Octal integer
x, X	Hexadecimal integer
s	String
t, T + suffix	Date / time
b	Boolean

Flag	Meaning
-	Left-align
+	Always show sign
0	Pad with zeros
,	Group thousands
(	Parentheses for negatives

String Formatting



How do width and precision affect the output?

```
1 double wind = 21.4532;  
2 System.out.println(String.format(Locale.US, "%2.2f km/h", wind));  
3 System.out.println(String.format(Locale.US, "%8.2f km/h", wind));  
4 System.out.println(String.format(Locale.US, "%08.2f km/h", wind));
```

Java

String Formatting

21.4532 formatted with Locale.US

%2.2f

2	1	.	4	5
---	---	---	---	---

%8.2f

			2	1	.	4	5
--	--	--	---	---	---	---	---

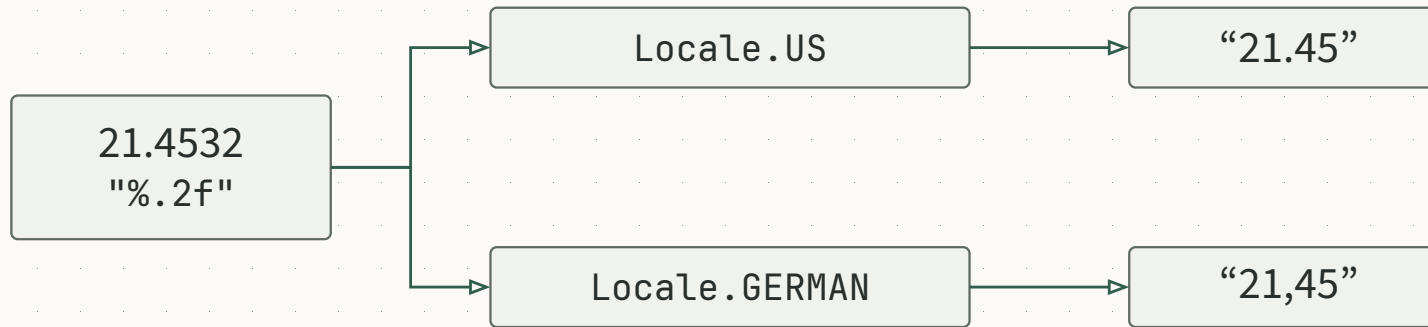
%08.2f

0	0	0	2	1	.	4	5
---	---	---	---	---	---	---	---

Width includes the decimal separator; it never truncates digits

 = one space

Formatting and Locale



Without an explicit locale, format uses the default formatting locale

Formatting and Locale

```
1 double wind = 21.4532;  
2 System.out.println(String.format(Locale.US, "%.2f km/h", wind));  
3 System.out.println(String.format(Locale.GERMAN, "%.2f km/h", wind));
```

Java

Output:

```
1 21.45 km/h  
2 21,45 km/h
```

text

3. Arrays

Arrays: C and Java

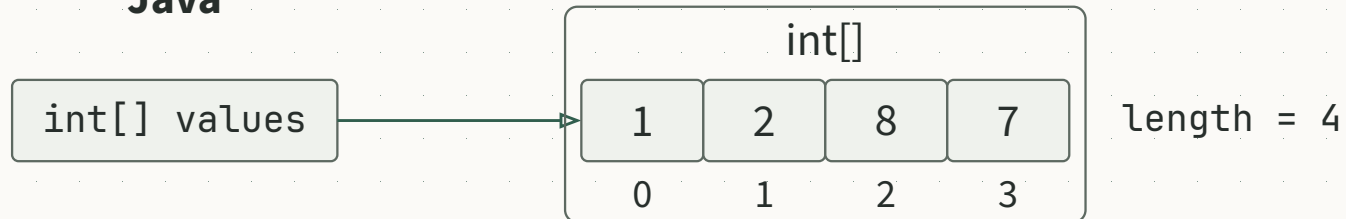
C

Pointer to first element



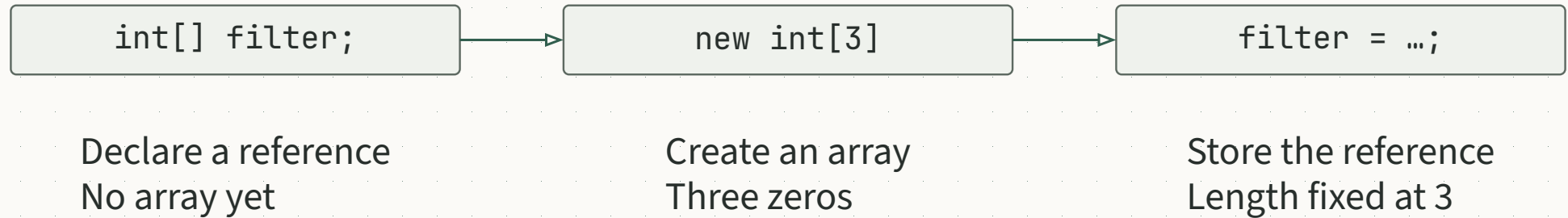
Program manages storage and bounds

Java



Reference to object · runtime manages memory

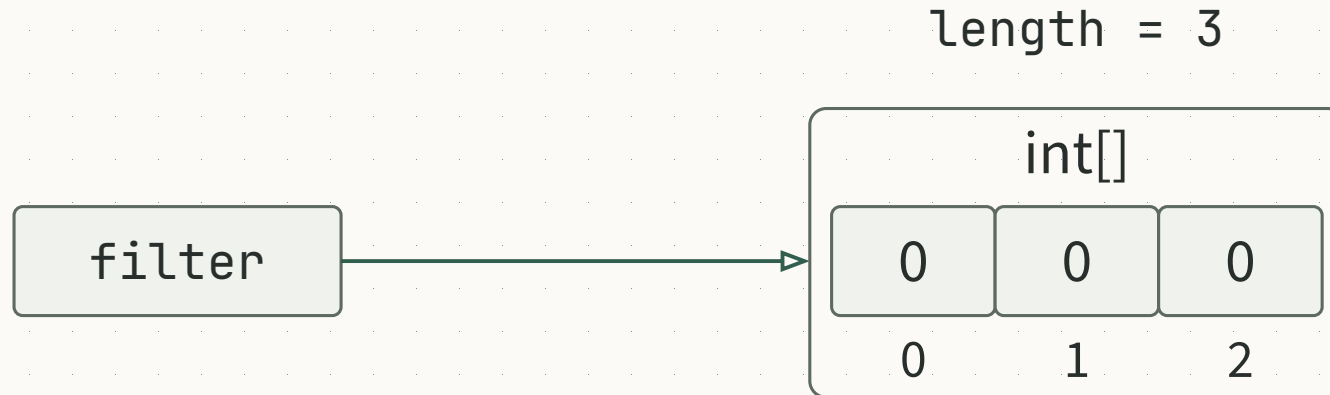
Creating Arrays



Use brackets after the type: `int[] filter`, not `int filter[]`.

Creation with `new`

```
1 int[] filter = new int[3];
```

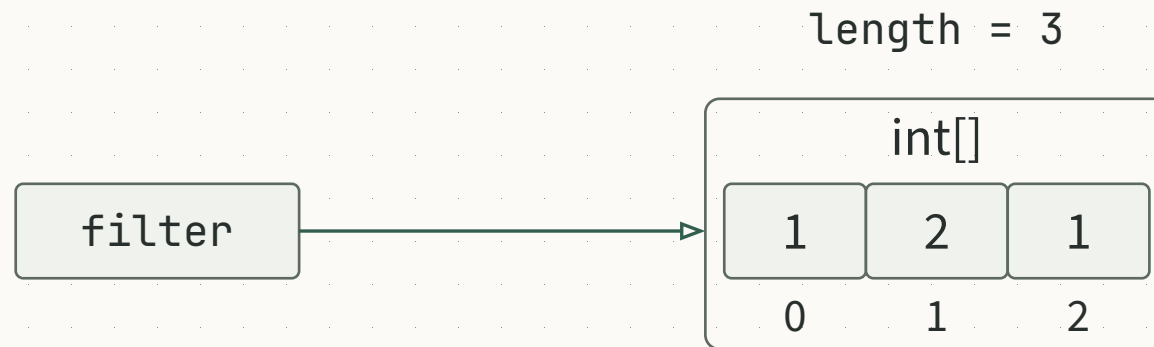
Java

Default elements: `0`, `0.0`, `false`, `'\u0000'`, or `null`, depending on type.

Assigning Elements

```
1 int[] filter = new int[3];  
2 filter[0] = 1;  
3 filter[1] = 2;  
4 filter[2] = 1;
```

Java



Array Initializers

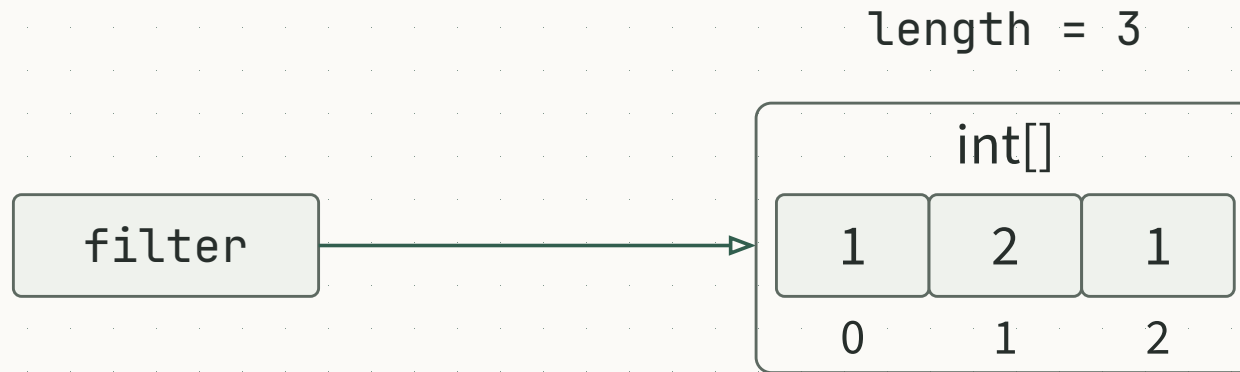
These alternatives create the same contents:

```
1 int[] filter = {1, 2, 1};
```

Java

```
1 int[] filter = new int[] {1, 2, 1};
```

Java



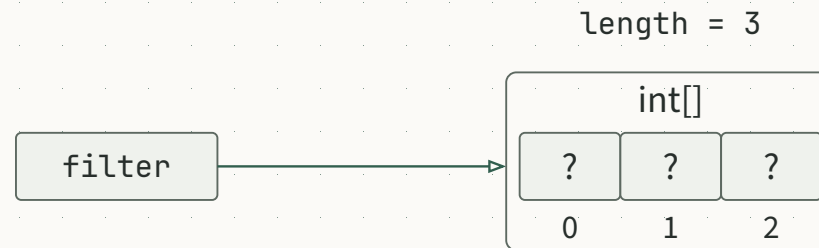
Array Length and Indices



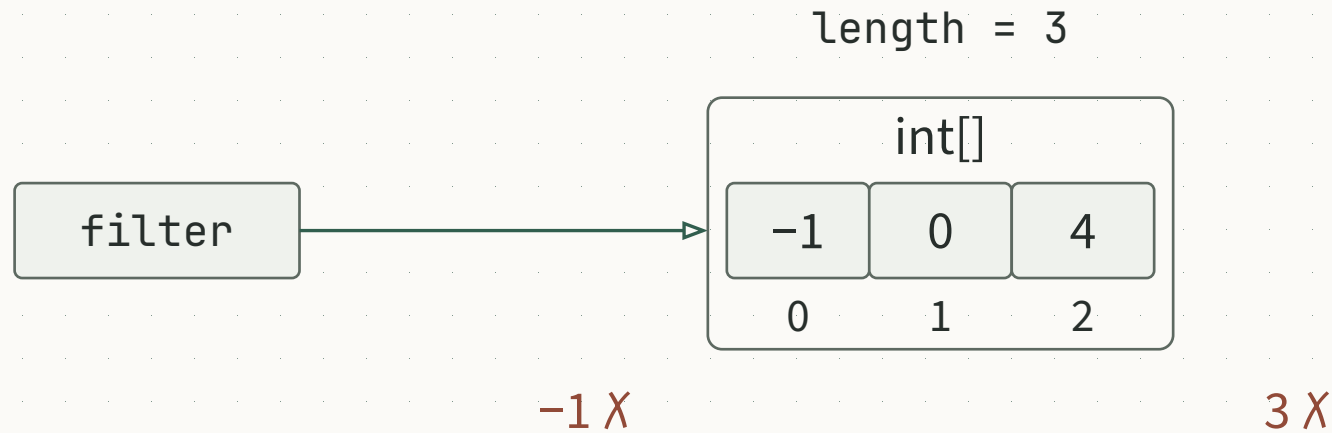
Which values does the loop store?

```
1 int[] filter = new int[3];  
2 for (int i = 0; i < filter.length; i++) {  
3     filter[i] = i * i;  
4 }
```

Java



Array Length and Indices



Valid indices: $0 \leq \text{index} < \text{length}$

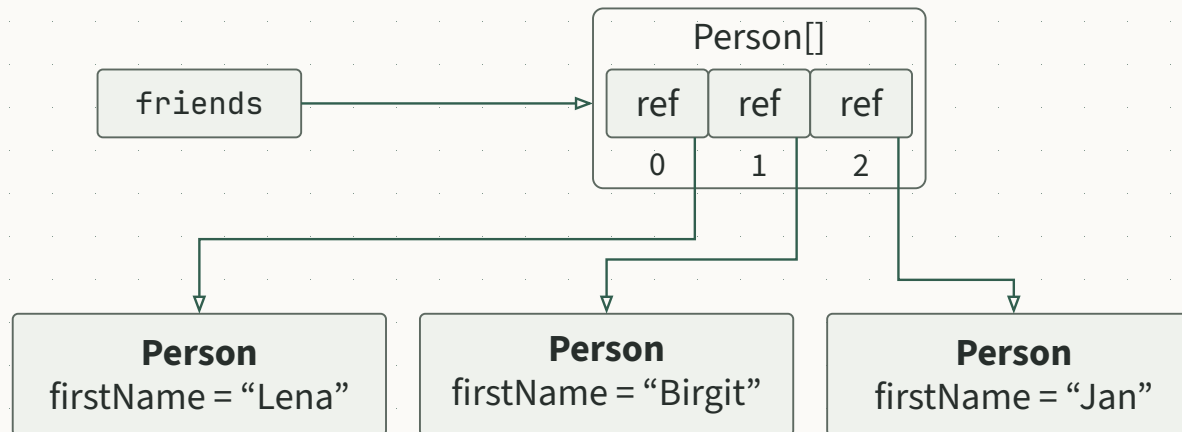
Otherwise: `ArrayIndexOutOfBoundsException`

`filter[-1]` and `filter[3]` are outside this array.

Arrays of Objects

```
1 Person[] friends = new Person[3];  
2 friends[0] = new Person("Lena", "Jensen");  
3 friends[1] = new Person("Birgit", "Meyer");  
4 friends[2] = new Person("Jan", "Schmidt");
```

Java



Array elements store references, not the Person objects

Array References

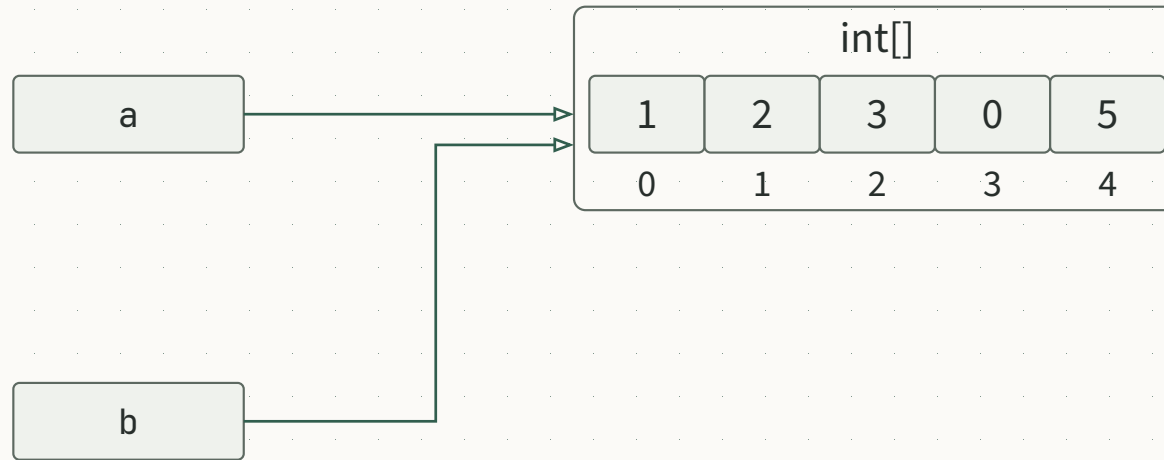


What will this print?

```
1 int[] a = {1, 2, 3, 4, 5};  
2 int[] b = a;  
3 b[3] = 0;  
4 System.out.println(b[2]);  
5 System.out.println(a[3]);
```

Java

Array References



`b[3] = 0` changes the array seen through both references

Output: 3, then 0.

Array References

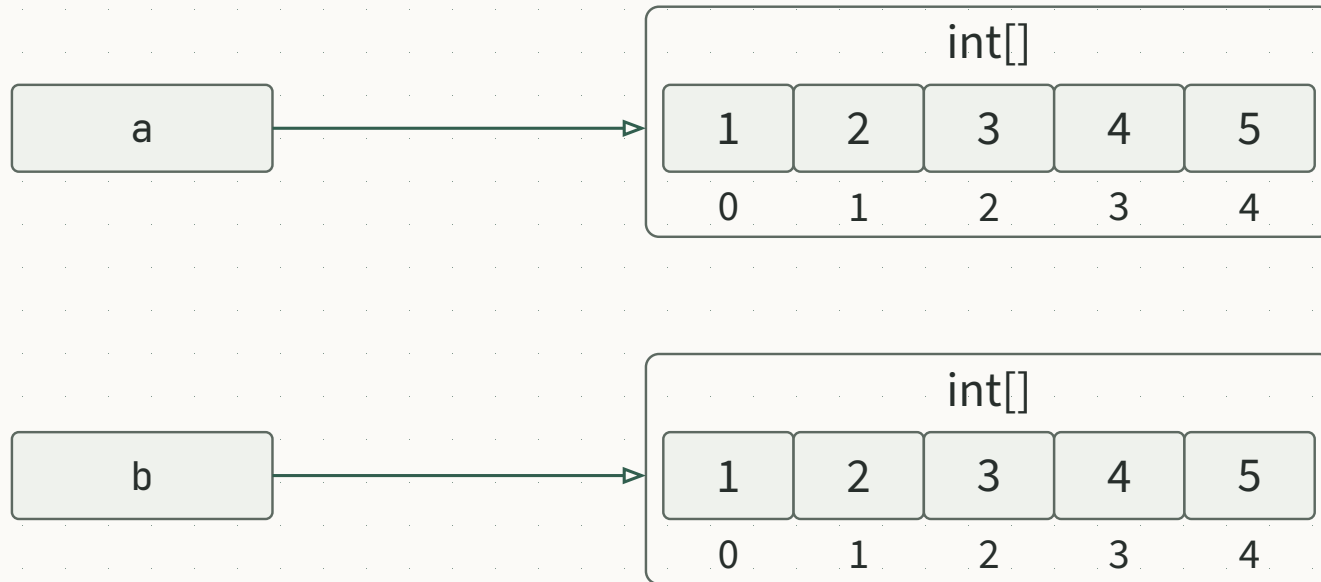


What will this print?

```
1 int[] a = {1, 2, 3, 4, 5};  
2 int[] b = {1, 2, 3, 4, 5};  
3 System.out.println(a == b);
```

Java

Array References



`a == b` → false: equal contents, different objects

Returning Arrays



What will this print?

```
1  static int[] createSortedArray(int a, int b) {  
2      if (a < b) {  
3          return new int[] {a, b};  
4      } else {  
5          return new int[] {b, a};  
6      }  
7  }  
8  
9  public static void main(String[] args) {  
10     System.out.println(createSortedArray(7, 4)[1]);  
11 }
```

Java

Sorting an Array



Task 3

Write a method that sorts an `int[]` in ascending order. Test it with `{10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 7}`.

Sorting an Array

```
1  public static void sort(int[] a) {  
2      int i = 0;  
3      while (i < a.length - 1) {  
4          if (a[i] > a[i + 1]) {  
5              int temp = a[i];  
6              a[i] = a[i + 1];  
7              a[i + 1] = temp;  
8              if (i > 0) i--;  
9          } else {  
10             i++;  
11         }  
12     }  
13 }
```

Java

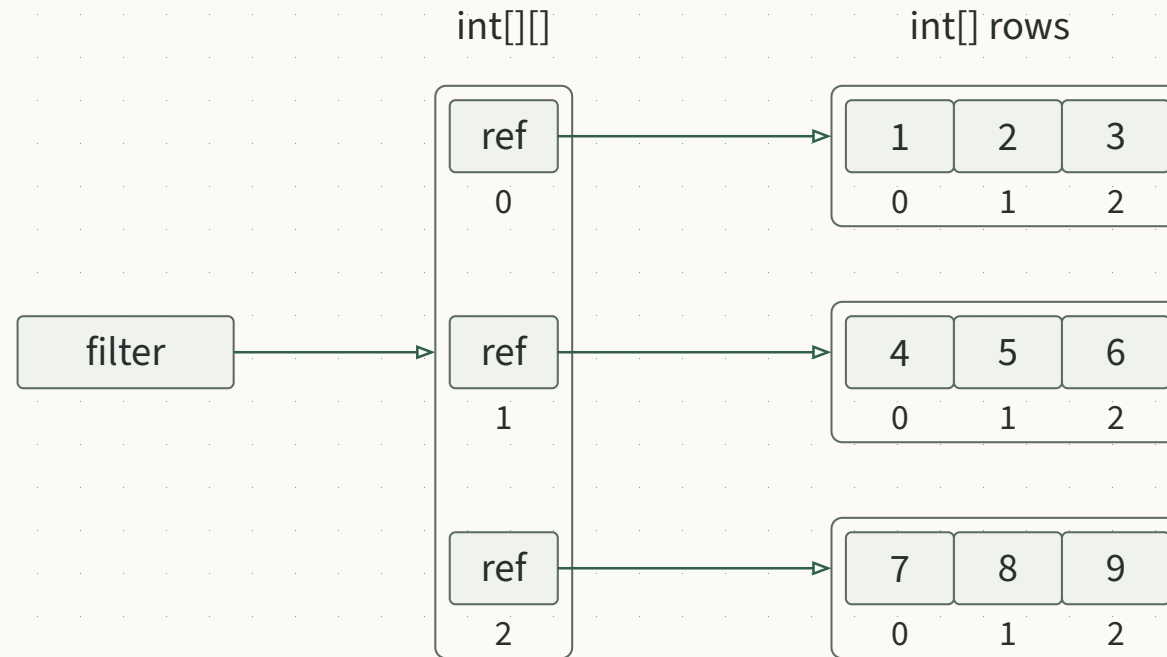
4. Multidimensional Arrays

Arrays of Arrays

```
1 int[][] filter = {  
2     {1, 2, 3}, {4, 5, 6}, {7, 8, 9}  
3 };
```

Java

Arrays of Arrays

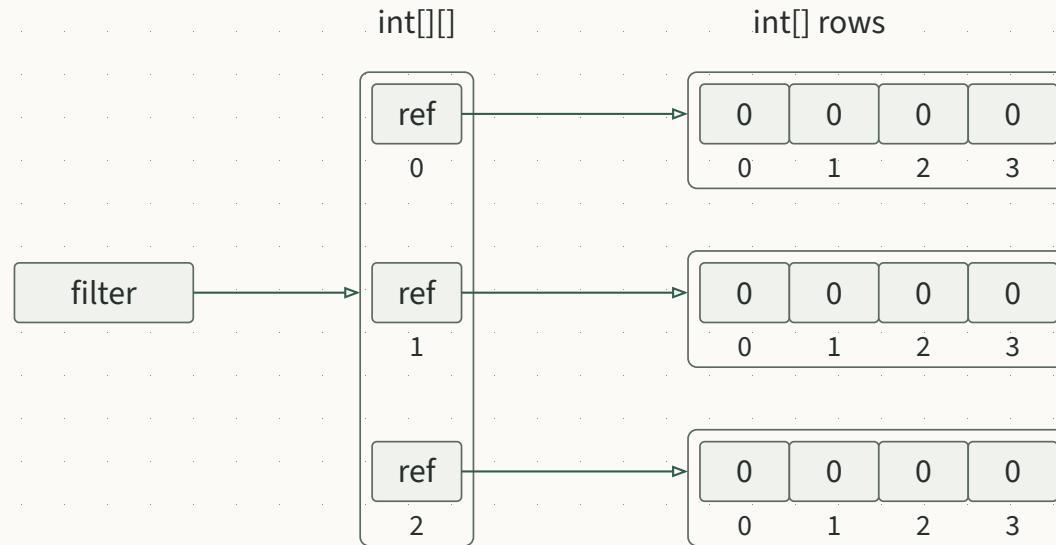


Each outer element refers to a row array

Arrays of Arrays

```
1 int[][] filter = new int[3][4];
```

Java



Each outer element refers to a row array

Arrays of Arrays

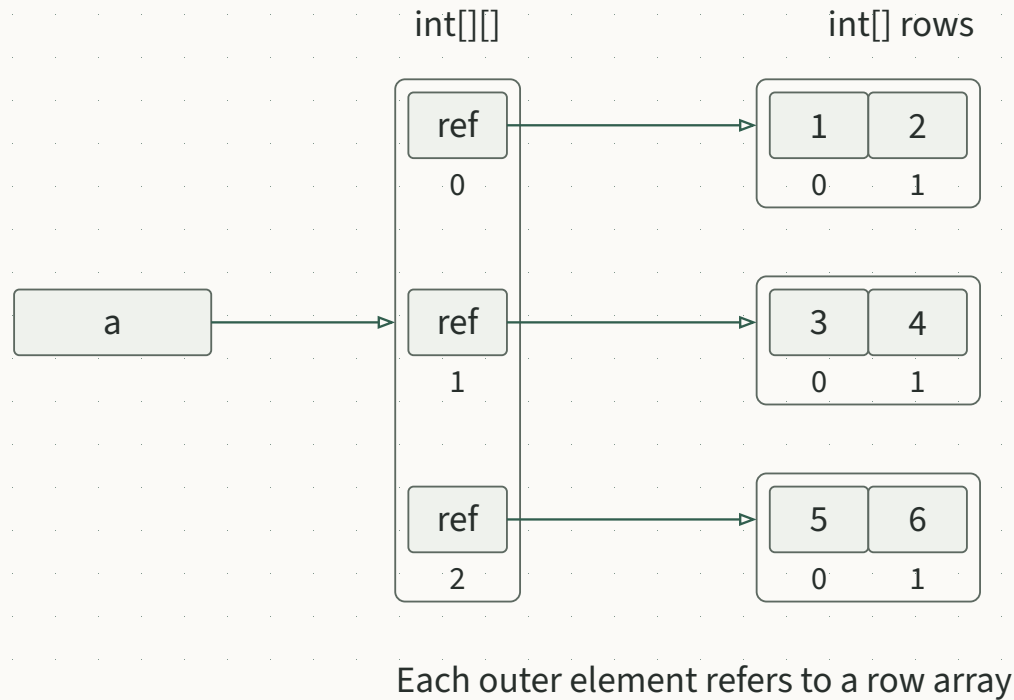


What will this print?

```
1 int[][] a = {{1, 2}, {3, 4}, {5, 6}};  
2 System.out.println(a.length);  
3 System.out.println(a[2].length);  
4 System.out.println(a[1][1]);  
5 System.out.println(a[2][0]);
```

Java

Arrays of Arrays



Output: 3, 2, 4, 5.

Row References and Copied Values

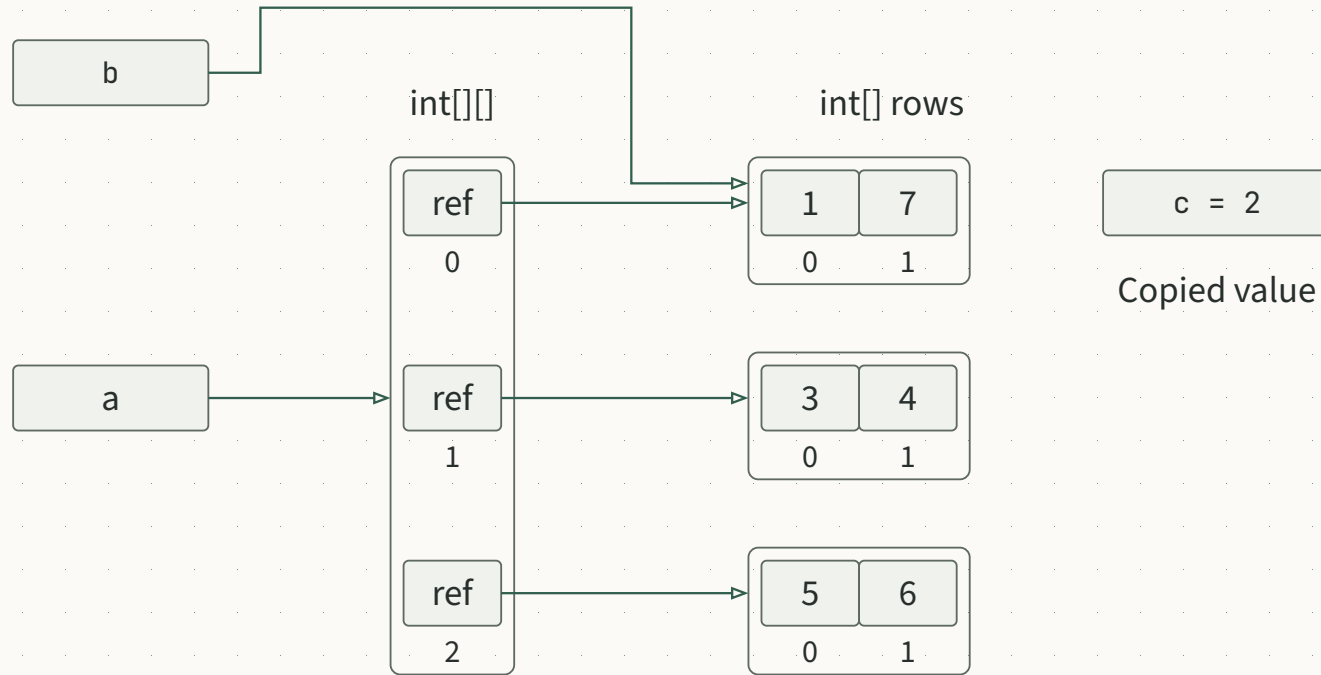


What will this print?

```
1 int[][] a = {{1, 2}, {3, 4}, {5, 6}};  
2 int[] b = a[0];  
3 int c = b[1];  
4  
5 b[1] = 7;  
6 System.out.println(a[0][1]);  
7 System.out.println(c);
```

Java

Row References and Copied Values



`b[1] = 7` changes `a[0][1]`; `c` stays 2

Jagged Arrays



Task 4

Use a for loop to create three rows with lengths 1, 2, and 3.

```
1 int[][] a = new int[3][];
```

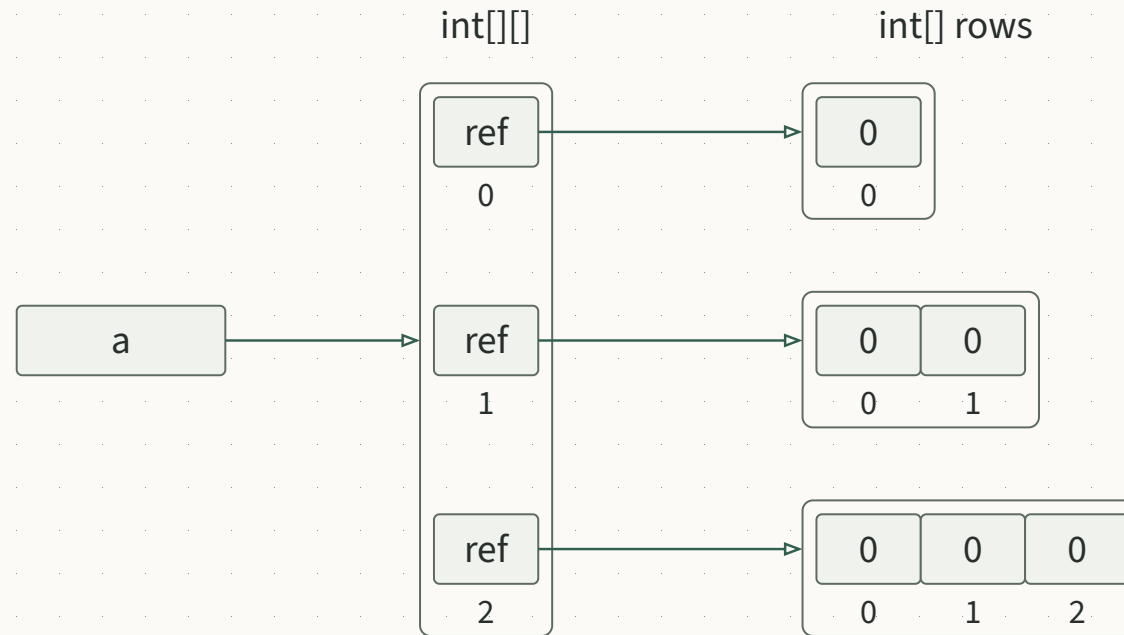
Java

Jagged Arrays

```
1 int[][] a = new int[3][];  
2 for (int i = 0; i < a.length; i++) {  
3     a[i] = new int[i + 1];  
4 }
```

Java

Jagged Arrays

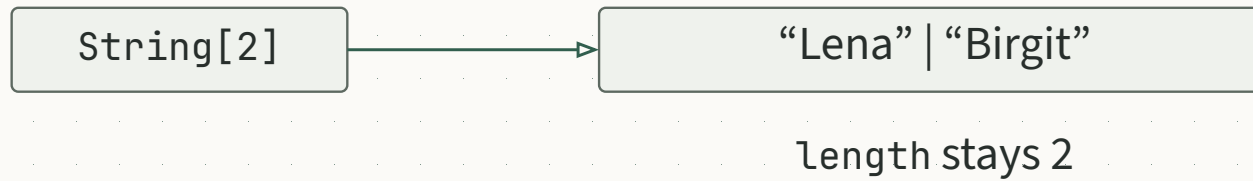


Each outer element refers to a row array

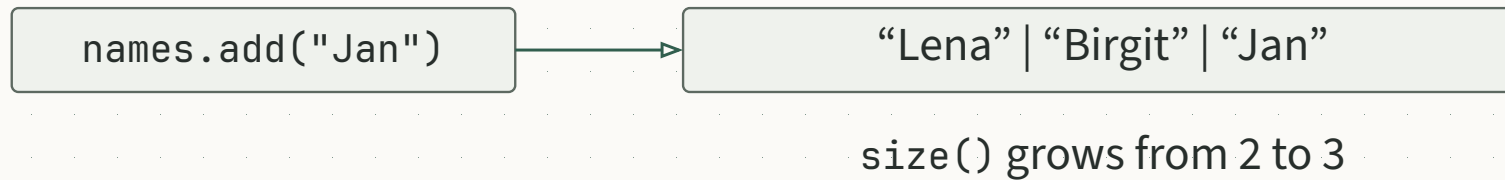
5. Lists

Array vs. ArrayList

Array



ArrayList



Creating an ArrayList

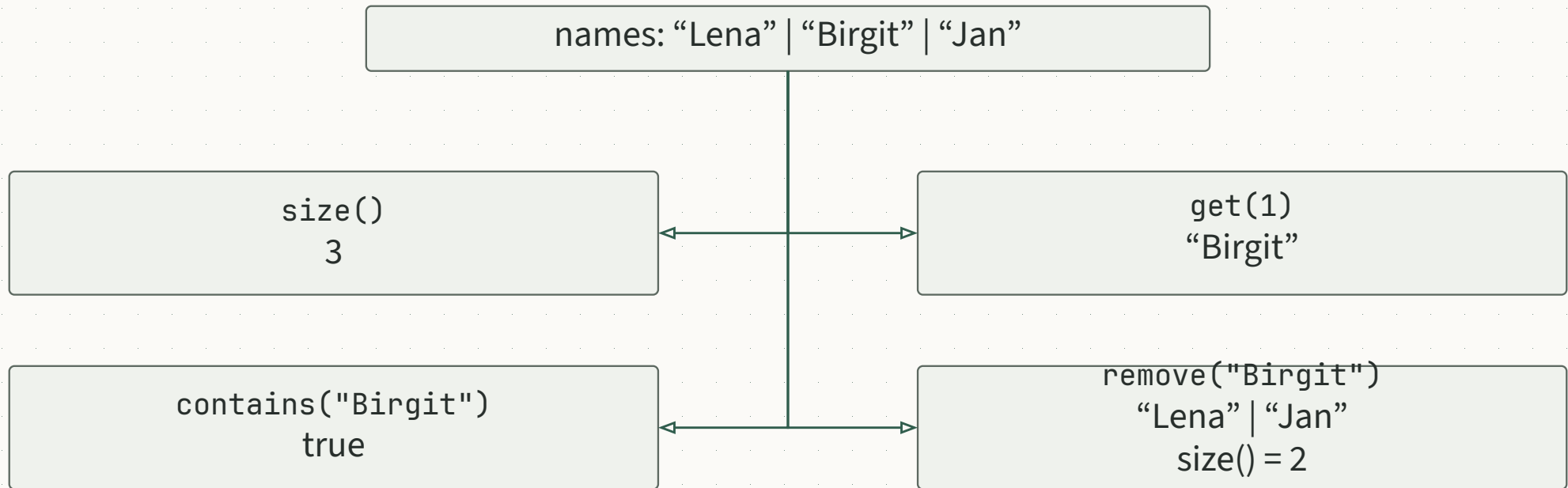
The type in angle brackets specifies the element type.

```
1  import java.util.ArrayList;
2
3  public class ArrayListDemo {
4      public static void main(String[] args) {
5          ArrayList<String> names = new ArrayList<>();
6          names.add("Lena");
7          names.add("Birgit");
8          names.add("Jan");
9          names.add(new String("Jan"));
10     }
11 }
```

Java

Lists allow duplicates. This list has four elements.

ArrayList Methods



ArrayList Methods

```
1  ArrayList<String> names = new ArrayList<>();
2  String birgit = "Birgit";
3  names.add("Lena");
4  names.add(birgit);
5
6  for (int i = 0; i < names.size(); i++) {
7      System.out.println(names.get(i));
8  }
9  if (names.contains(birgit)) {
10     names.remove(birgit);
11 }
```

Java

6. foreach Loop

foreach Loop

Use foreach when you need every element, but not its index.

```
1 for (DataType element : arrayOrList) {  
2     // Use element  
3 }
```

Java

foreach Loop

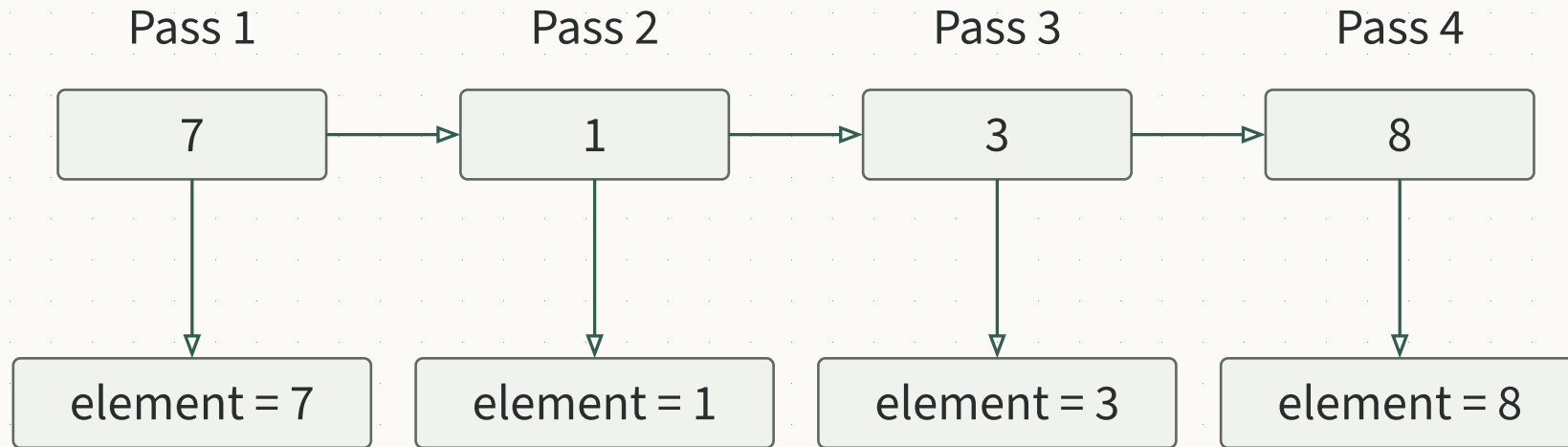


What will this print?

```
1 int[] a = {7, 1, 3, 8};  
2 for (int element : a) {  
3     System.out.println("Element: " + element);  
4 }
```

Java

foreach Loop



Visit each element in order; no index variable needed

For primitive elements, the loop variable receives a copy of each value.

Task: Average



Task 5

Use foreach to calculate the average of a non-empty `double[]`.
Write a program that calls the method.

Task: Average

```
1 static double average(double[] numbers) {  
2     double sum = 0.0;  
3     for (double number : numbers) {  
4         sum += number;  
5     }  
6     return sum / numbers.length;  
7 }
```

Java

Precondition: numbers is non-null and non-empty.

Task: Average

```
1 public static void main(String[] args) {  
2     double[] a = {1.43, 2, .2, 6.32, 7.1, 8.1};  
3     System.out.println("Average = " + average(a));  
4 }
```

Java

7. Wrapper Classes & `Math` Class

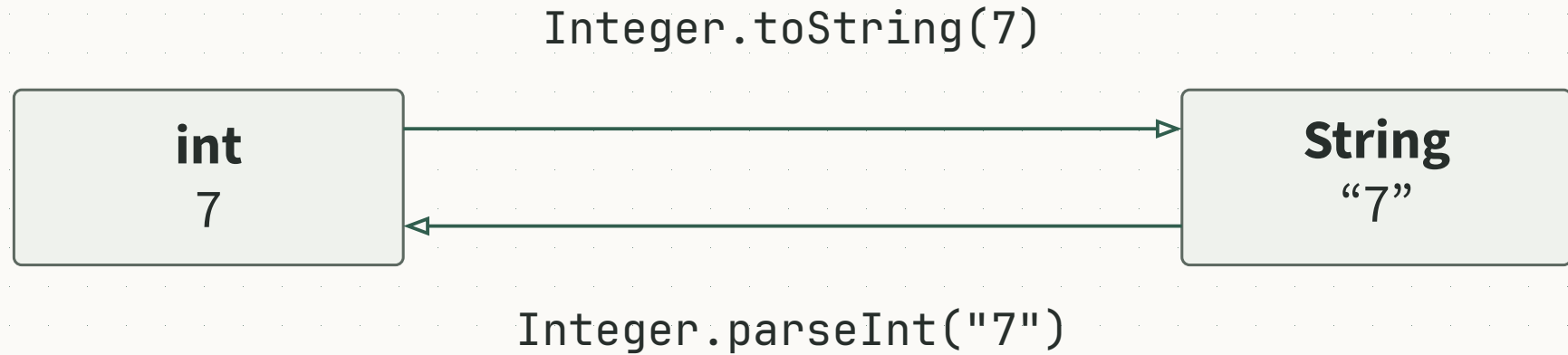
Wrapper Classes

Wrappers represent primitive values as objects and provide conversion methods.

Primitive	Wrapper	Primitive	Wrapper
boolean	Boolean	long	Long
byte	Byte	char	Character
short	Short	float	Float
int	Integer	double	Double

Generic types require reference types: `ArrayList<Integer>`, not `ArrayList<int>`.

Numbers and Strings



Invalid integer text → `NumberFormatException`

Numbers and Strings

```
1 int a = 7;  
2 Integer b = Integer.valueOf(a);  
3 String c = b.toString();
```

Java

Or use the class method directly:

```
1 String text = Integer.toString(7);  
2 int number = Integer.parseInt(text);
```

Java

Boxing and Unboxing

Boxing: `Integer.valueOf(24)`



Unboxing: `object.intValue()`

Automatic: `Integer object = 24; → int value = object;`

Unboxing null throws `NullPointerException`

Boxing and Unboxing

Explicit conversion

```
1 Integer object = Integer.valueOf(24);  
2 int value = object.intValue();
```

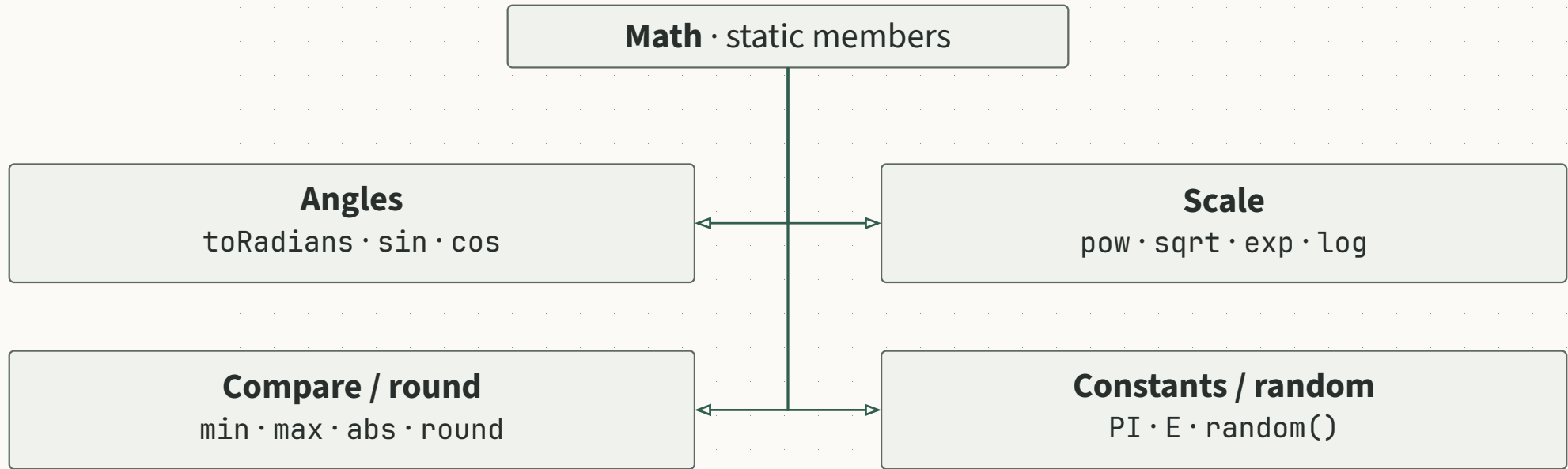
Java

Automatic conversion

```
1 Integer object = 24;  
2 int value = object;
```

Java

The Math Class



The Math Class

Trigonometric methods use radians.

```
1 double angleDeg = 127.5;  
2 double angleRad = Math.toRadians(angleDeg);  
3 System.out.printf("cos(%.2f) = %.2f\n",  
4     angleRad, Math.cos(angleRad));
```

Java

8. License Notice

Attribution

- Shared under CC BY-NC-SA 4.0 and the applicable public license.
- Based on teaching material by Prof. Dr. Marc Hensel.
- Text, diagrams, and layout adapted. Original material supplied privately; no public source link is available.